Polić's Roadmap — Frontend

Oct 16, 2026 · @Marin

What you do, phase by phase, from today until the June 2027 launch. It tells you *what* to do, *what to learn* for it and *how you know it's done*, never how to solve it. The questions in each phase are for you to answer yourself, ideally in an ADR. Dates follow the MVP plan, and every date shared with a teammate is the same in their roadmap.

How to use this roadmap

Each phase has the same four parts:

- **Do:** the tasks, in a sensible order. A date means someone else is waiting for it. Turn each task into a GitHub issue when the phase starts.
- **Learn first:** what to read before starting, always official docs first.
- **Questions to answer yourself:** design decisions with no single right answer. Write your answer and its reasons in a short ADR in `docs/adr/`. That's where most of your learning will happen.
- **Done when:** how you know you can move on. If it isn't checked, it isn't done.

Your area: every page students, landlords and moderators see. The page routes, templates, CSS, HTMX and both maps live in `app/web/`. You decide how pages look and behave, and you review every change to `app/web/`.

Not yours, so you never build it twice: what happens underneath a page. Each page calls a function someone else owns and shows the result:

- Listings, accounts, photos, search and reviews are Marin's.
- Saved searches and unsubscribing are Bob's.
- The fair-rent estimate is Sigi's.

If a page needs a rule (may this person see this? is this price valid?), the rule belongs in their function, not in your template.

You have the most pages, and most hand-offs end with you. Build every page first with fake data typed into your route, so you never sit waiting; switch to the real function the day it lands. If you're falling behind, say so at the sync early, so Marin can cut scope instead of you working through the holidays.

Your week: about 5 hours: ~4 h on your own issues, ~1 h for the sync and for reviewing PRs that touch your folder.

Rules that keep us in sync

These nine rules are word for word the same in all four roadmaps. They exist so that no two people build the same thing and nobody waits in silence.

1. **One owner per task.** If something isn't in your roadmap, it isn't yours yet: ask Marin before you start it.
2. **Pages display, modules decide.** Rules (who may do what, what's valid, what matches, what a flat should cost) live in the owner's module. Pages call them and show the result.
3. **One definition of "matches the filters":** Marin's search. Alerts reuse it; nobody writes a second one.
4. **Database changes only through migrations, reviewed by Marin.** Pull `main` and say so in Discord before you create one. One open migration PR at a time.
5. **Shared files belong to Marin:** `docker-compose.yml`, `pyproject.toml`, `.env.example`, `app/main.py`, `.github/`. Anyone may change them in a PR; Marin reviews it.
6. **Changing something another person's code calls?** Agree it with them first, and ask them to review the PR.
7. **Blocked?** Say so the same day, in #help or at the sync. Meanwhile, keep going with fake data.
8. **Secrets never go in Git, Discord or AI tools.** `.env.example` holds setting names only, never values.
9. **A date in the hand-off table is a promise to a teammate.** If it will slip, tell them as soon as you know, not on the day.

Your hand-offs

Every date where your work meets someone else's. The same rows, with the same dates, are in their roadmaps. Each row becomes an issue on the board. Almost every row ends with you, a week before the page that needs it is due.

Due	From → to	What
21 Oct	Sigi → everyone	Rent survey live, with a link everyone shares
22 Oct	Marin → everyone	One-command local setup (database + minimal app) and README steps to run it
25 Oct	Marin → everyone	Phase 1 issues on the board, written from these roadmaps and assigned
25 Oct	you → Marin	Paper sketches of every MVP page, each noting what it shows (input for the data model)
29 Oct	Marin → you	The app set up so pages, templates and CSS can be added inside `app/web/` alone
29 Oct	Sigi → you, Marin	The fixed neighbourhood list: names, and in words where each one is
1 Nov	Marin → everyone	Data model v1 agreed (ER diagram + ADR), including the search filters and their units
8 Nov	Marin → everyone	Core listing functions: create, get one, list with the price and rooms filters
8 Nov	Marin → you, Bob	Test database setup: how to write a test that uses the database
15 Nov	Marin → you	How logging in works, which decides the account pages
15 Nov	Marin → everyone	CI runs everyone's tests on every pull request
15 Nov	Marin → everyone	Every merge to `main` deploys to the live server
15 Nov	Sigi → you	`estimate_rent()` placeholder with agreed inputs and output; always answers "not enough data"
15 Nov	Sigi → everyone	Seed script that fills a database with fake listings
22 Nov	Marin → you	Account functions: sign up with a university email, confirm the code, log in, log out, tell who is logged in
22 Nov	you → Bob	The address of a listing page, for links in alert emails

Due	From → to	What
29 Nov	Marin → you, Bob	Roles and permission checks in one place (student, verified student, landlord, moderator), and protection for forms
29 Nov	Marin → you	Listing functions complete: edit, mark as taken, a landlord's own listings, location from the map pin (checked to be in Osijek), neighbourhood from Sigi's list
6 Dec	Marin → you	Photos: what is allowed, where they are kept, how a page shows one
6 Dec	Marin → you, Bob	Faculty locations and search by distance: the one definition of "matches the filters", including how results are split into pages
6 Dec	Bob → you	Saved-search functions: create (verified students only), list a student's own, delete
6 Dec	Sigi → you	Fair-rent estimate v1 (statistics), same inputs and output as the placeholder
13 Dec	Marin → you	Review functions: create, list for a flat, report, moderator hide / restore / delete
13 Dec	Bob → you	Unsubscribe: what the link in each alert email looks like, and the function behind it
13 Dec	Sigi → you	Final MVP estimate (the better of statistics and regression), same inputs and output
17 Jan	Marin → you	Account deletion function and the privacy policy text
17 Jan	you → Marin	Screenshots of the main pages, for the README
17 Jan	Sigi → you	One sentence explaining how the estimate is made, for the listing page

Every page you build

The full list, so nothing is missing and nobody else builds one. "Uses" is whose functions the page calls, with the date they land; "Due" is when the page works on the live site.

#	Page	What it does	Uses	Due
1	Base layout	Header, navigation, footer; works on a phone from the start	—	8 Nov
2	Listing results	The list of listings	Marin, 8 Nov	15 Nov
3	Listing page	One listing: details, contact, fair-rent estimate	Marin, 8 Nov · Sigi, 15 Nov	22 Nov
4	Results map	Every listing as a marker on a Leaflet map	Marin, 8 Nov	22 Nov
5	Account pages	Sign up, confirm the code, log in, log out; who's logged in, in the header	Marin, 22 Nov	29 Nov
6	Listing form	Create and edit a listing: map pin, neighbourhood dropdown	Marin, 29 Nov	6 Dec
7	My listings	A landlord's own listings: edit, mark as taken	Marin, 29 Nov	6 Dec
8	Search filters	Price, rooms, faculty, distance; results and map update without a reload	Marin, 6 Dec	13 Dec
9	Photos	Upload in the listing form; shown on the listing page	Marin, 6 Dec	13 Dec
10	Saved searches	"Save this search" on the results; a page to see and delete them	Bob, 6 Dec	20 Dec
11	Reviews	Reviews on the listing page, the review form, a report button	Marin, 13 Dec	20 Dec
12	Moderation	Reported reviews: hide, restore, delete	Marin, 13 Dec	20 Dec
13	Unsubscribe	Where the link in an alert email lands	Bob, 13 Dec	20 Dec

#	Page	What it does	Uses	Due
14	Error and empty states	Not found, something went wrong, nothing matches	—	10 Jan
15	Privacy and account	Privacy policy page, delete-account page	Marin, 17 Jan	24 Jan

Phase 0 — now until 25 October: setup and learning

Goal: you can run the project locally, you know the workflow, and every MVP page exists on paper.

Do

1. **Everyone's Phase 0:** install Python, Git, VS Code and Docker; finish the #git-guide; do the official FastAPI tutorial up to "SQL databases".
2. Learn HTML and CSS basics: page structure, forms, layout, and responsive design for phones.
3. Learn how Jinja templates work: inheritance, loops, conditions, escaping.
4. Read the HTMX docs and try a few of their examples on your own machine.
5. **Sketch every page** in the page table on paper, each noting what information it shows. Show them at the sync and give them to Marin by 25 Oct: they're an input for his data model.
6. Share Sigi's rent survey once it's live (21 Oct) and collect answers.
7. Run the app locally following only Marin's README (ready 22 Oct). Tell Marin every place you got stuck.
8. Merge your first reviewed PR (for example, adding your name to the README).
9. Read your Phase 1 issues when Marin posts them (25 Oct) and tell him what's unclear or missing.

Learn first

- MDN: Learn web development: HTML, CSS, the web forms module, and responsive design.
- Jinja template designer docs.
- HTMX docs and examples.

Questions to answer yourself

- Which pages will students mostly open on a phone, and what does that change in your sketches?
- What's the one thing a student must see first on a listing page?
- What's the difference between a full page reload and an HTMX update, from the user's point of view?

Done when

- [ ] Every page in the table is sketched, shown at the sync and handed to Marin
- [ ] You ran the app locally with one command, following only the README
- [ ] You merged one reviewed PR

Phase 1 — 26 October to 22 November: real listings on real pages

Goal: the live site shows listings from the database on your pages, on a phone and on a laptop.

Do

1. **Check the data model** (agreed by 1 Nov): make sure it gives every page in your sketches the information it shows. Say so before it's agreed, not after.
2. **Base layout** (by 8 Nov): header, navigation, footer, mobile-friendly from the start. By 29 Oct, Marin sets up the app so you can add pages inside `app/web/` without editing his files.
3. **CSS decision:** plain CSS or a small CSS framework (the MVP plan leaves it to you). Write the ADR before the base layout grows.
4. **Listing results** (by 15 Nov): first with fake data typed into your route; switch to Marin's listing functions when they land (8 Nov). Sigi's seed script (15 Nov) fills your local database with fake listings.
5. **Listing page and results map** (by 22 Nov): one listing with its details, and every listing as a marker on a Leaflet map. Filters come in Phase 2.
6. **Fair-rent estimate on the listing page** (by 22 Nov): agree the inputs and output of `estimate_rent()` with Sigi. His placeholder lands 15 Nov and always answers "not enough data"; the page must show that properly.
7. **Give Bob the address of a listing page** by 22 Nov, for the links in his alert emails.

8. **Plan the account pages:** Marin tells you how logging in works by 15 Nov. Sketch the pages that follow from it now; you build them first thing in Phase 2.
9. **Tests** for your pages, using Marin's test database setup (8 Nov). From 15 Nov, CI runs your tests on every PR, and every merge goes live automatically.

Learn first

- FastAPI: templates, static files and testing.
- Leaflet quick start and the OpenStreetMap tile usage policy.
- Jinja's autoescaping (in the template designer docs) and cross-site scripting (OWASP).

Questions to answer yourself

- What happens if a landlord types `<script>` into a listing description? Check it, don't assume.
- What does the listing page show when a listing has no photo, no estimate, or is already taken?
- What does the OpenStreetMap tile policy ask of us, and how will you meet it?
- How does the map look with 3 listings? With 300?
- Should Leaflet's files come from a CDN or from our own static folder? What does each choice mean?

Done when

- [ ] The live site shows fake listings on the results page, the listing page and the map
- [ ] Every page works on a phone screen and on a laptop screen
- [ ] The listing page shows Sigi's "not enough data yet" properly
- [ ] Your page tests run in CI
- [ ] The CSS ADR is merged

Phase 2 — 23 November to 20 December: every MVP page

Goal: every MVP page works on the live site, even if rough. Each step below uses functions that land a week before it's due.

Do

1. **Account pages** (by 29 Nov): sign up, confirm the code, log in, log out, and who's logged in, in the header. Uses Marin's account functions (22 Nov). When his form protection lands (29 Nov), add it to every form you have.
2. **Listing form and My listings** (by 6 Dec): create and edit a listing, with a map where the landlord drops a pin and a dropdown of Sigi's neighbourhoods (his list is fixed since 29 Oct); a landlord's own listings with edit and mark as taken. Uses Marin's listing functions (29 Nov).
3. **Search filters and photos** (by 13 Dec): price, rooms, faculty and distance; results and map update together without a page reload. Photo upload in the listing form, photos on the listing page. Uses Marin's search (6 Dec) and photos (6 Dec).
4. **Saved searches, reviews, moderation, unsubscribe** (by 20 Dec): "Save this search" and the saved-searches page (Bob, 6 Dec); reviews on the listing page, the review form and a report button, and the moderation page (Marin, 13 Dec); the unsubscribe page (Bob, 13 Dec).
5. **Check the estimate:** when Sigi's real estimate lands (6 Dec, final version 13 Dec), the listing page must show a range or "not enough data yet" without you changing any code. If you have to change code, the agreement broke: tell Sigi.
6. **Tests** for every page.

Learn first

- FastAPI: form data and request files.
- The HTMX docs on triggers, targets and swapping.
- The Leaflet reference: markers and map events.
- Cross-site request forgery (OWASP): what Marin's form protection defends against.

Questions to answer yourself

- After a landlord submits a form with a mistake, what do they see? Do they lose what they typed?
- How does a landlord fix a pin dropped in the wrong place?
- When filters change, what happens to the back button and to a link a student copies to a friend?
- What does a logged-out visitor see when they open a page they may not use? (Marin's check decides whether they may; you decide what they see.)

- What does a student who isn't verified see where the review form would be?
- How many results fit on a phone screen, and how does a student get to the next ones?

Done when

- [ ] Pages 1–13 in the table work on the live site, on a phone and a laptop
- [ ] A landlord can post a listing with a pin, a neighbourhood and photos, edit it and mark it as taken, without help
- [ ] Filters combine correctly, and the map shows every result
- [ ] Every form carries Marin's protection
- [ ] For every decision a page shows, you can point to the function that made it

Holiday buffer + Phase 3 — 21 December to 24 January: usable on any phone

Holiday buffer (21 Dec – 3 Jan): only catch-up on unfinished Phase 2 pages. Nothing new. Rest counts as part of the plan.

Phase 3 goal: real students can use every page on their phones without help.

Do

1. **Error and empty states** (by 10 Jan): not found, something went wrong, no listings match, no reviews yet, no saved searches.
2. **Real phones** (by 17 Jan): test every page on at least two real phones, one of them small. Fix what breaks.
3. **Accessibility basics** (by 17 Jan): a label on every form field, enough contrast, everything usable with a keyboard, text alternatives for images.
4. **Screenshots** of the main pages for Marin's README, by 17 Jan.
5. **Privacy and account pages** (by 24 Jan): the privacy policy page and the delete-account page, using Marin's text and function (17 Jan).
6. **Estimate explanation** (by 24 Jan): show Sigi's one-sentence explanation of the estimate (17 Jan) on the listing page.
7. **Page speed** (by 24 Jan): measure how long the results page takes on a slow connection, and fix the biggest cause.

Learn first

- W3C's tips for developing accessible sites and the WCAG quick reference (levels A and AA only).
- Firefox DevTools: responsive design mode and network throttling.

Questions to answer yourself

- Can you use the whole site with only a keyboard?
- What does a student on a slow mobile connection wait for first?
- What happens to page speed when a listing has 20 large photos?
- When something breaks, what should the page say so the student isn't stuck?

Done when

- [ ] Every page has been tested on real phones
- [ ] Every form passes your accessibility checklist
- [ ] Every page has its error and empty states
- [ ] The privacy policy and delete-account pages are live
- [ ] Screenshots delivered to Marin

Watch out: the exam period starts around the end of January. If Phase 3 slips, cut scope rather than pushing into exams.

Pilot and launch — February to June

Goal: students and landlords use the site without anyone explaining it.

Do

1. Watch at least three pilot users, students and landlords, use the site without helping them. Write down where they hesitate.
2. Fix the biggest confusion first, then watch again.
3. Take your share of contacting landlords and recruiting pilot students, as split at the sync.
4. Take your turns on call, as agreed at the sync.
5. Propose your "Later" features (installable mobile version, analytics) only once the pilot shows they're needed.

Questions to answer yourself

- What did users try to click that isn't clickable?
- Which page do landlords give up on, and why?

Done when

- [ ] Every confusion you saw in the pilot is fixed, or in "Later" with a reason
- [ ] Your CV line is backed by numbers you can explain in an interview (pages, users, what you measured and improved)

Every week, and when you're stuck

- **Sync:** say what you finished, what's next and where you're stuck. Show a short demo when you can.
- **Reviews:** review PRs that request you within 48 hours. Ask questions instead of writing the fix.
- **Board:** keep your issues' status current. Hand-offs due this week come first.
- **Once per phase:** make at least one change outside your own area. Pick it with Marin at the phase planning, so it doesn't collide with the owner's work.

When you're stuck on something technical:

1. Write down exactly what you expected and what happened instead.
2. Make the smallest possible example that shows the problem.
3. Read the official docs for that exact thing.
4. Ask in #help, the way the post guidelines describe.
5. Only then, follow the AI policy: ask for an explanation, not a solution.

Everyone on the team is learning this at the same time. Nobody expects you to know it in advance; the team expects you to say early when you don't know something yet.